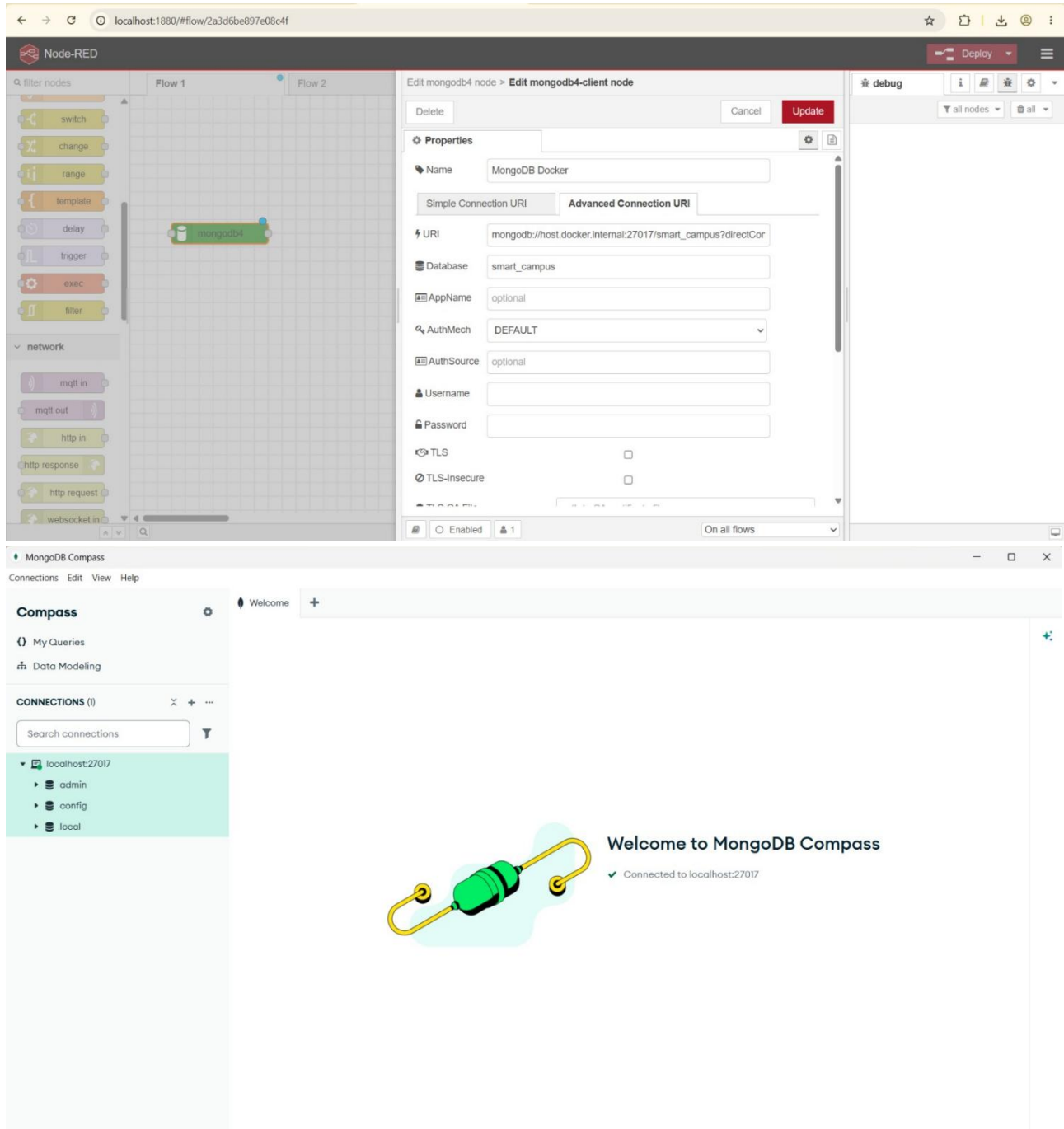


EXERCISE 6: DATABASE FOUNDATIONS

Step 1: Deploy MongoDB Locally via Docker

□ ● MongoDB 32df469a784e [mongodb/](#) 27017:27017 ↗ 3.02% 130.8    

Step 2 & Step 3: Install MongoDB Nodes and Configure the Database Connection in Node-RED



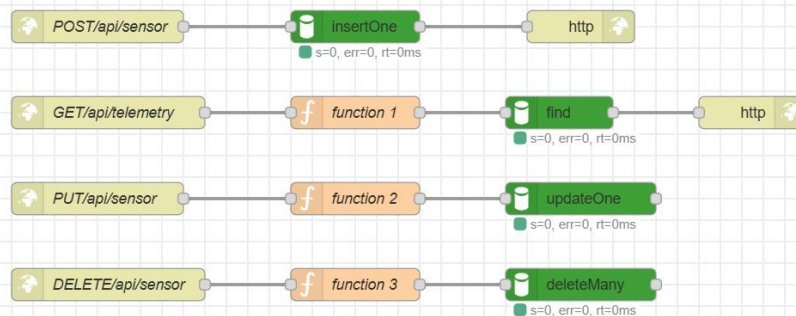
The screenshot displays the Node-RED web interface in a browser window. The main workspace shows a flow with a 'mongodb' node. The right-hand pane is open to the 'Edit mongodb4-client node' configuration. The 'Name' is set to 'MongoDB Docker'. The 'URI' is 'mongodb://host.docker.internal:27017/smart_campus?directConnect=true'. The 'Database' is 'smart_campus'. The 'AuthMech' is set to 'DEFAULT'. The 'AuthSource' is 'optional'. The 'Username' and 'Password' fields are empty. The 'TLS' and 'TLS-Insecure' checkboxes are unchecked. The 'Enabled' checkbox is checked. The 'On all flows' dropdown is set to 'On all flows'.

Below the Node-RED interface, the MongoDB Compass application is shown. The 'Connections' tab is active, displaying a list of connections. The connection 'localhost:27017' is selected, showing a list of databases: 'admin', 'config', and 'local'. The 'Welcome' message is displayed, stating 'Welcome to MongoDB Compass' and 'Connected to localhost:27017'.

Step 4: Upgrading REST APIs to execute CRUD Operations

The first screenshot shows a POST request to `http://localhost:1880/api/sensor` with a JSON body: `{ "temperature": 25, "humidity": 60, "device_id": "ESP32_01" }`. The response is a 200 OK status with a JSON body: `{ "acknowledged": true, "insertedId": "6a06af4fb8229a42a343600e" }`.

The second screenshot shows a GET request to `http://localhost:1880/api/telemetry` with a JSON body: `{ "temperature": 25, "humidity": 60, "device_id": "ESP32_01" }`. The response is a 200 OK status with a JSON body: `[ { "_id": "6a06af4fb8229a42a343600e", "temperature": 25, "humidity": 60, "device_id": "ESP32_01" } ]`.



Part 5: Discussion & Architectural Analysis

Question 1: Dynamic Schema Generation

A **schema-less** (or more accurately, *schemaless*) database allows you to insert data without defining the structure, columns, or data types ahead of time. In traditional relational databases (like MySQL or PostgreSQL), you must explicitly run a CREATE TABLE statement defining exactly what fields exist and what data types they hold (e.g., temperature FLOAT). If a device sends an undefined field, the database rejects it. In a document-oriented database like MongoDB, data is stored as flexible documents (JSON/BSON). Each document can have entirely different fields, and the database adapts on the fly to whatever data you throw at it.

Why it's a game-changer for early IoT prototyping:

- **Frictionless Hardware Iteration:** In the early stages of IoT, your hardware configuration changes constantly.
- **Handling Heterogeneous Fleet Data:** If you are testing five different sensor models from different manufacturers, they will likely output data in slightly different JSON structures.
- **Speed to MVP (Minimum Viable Product):** You spend your engineering cycles writing firmware and building dashboards rather than managing database schemas and alignment layers.

Question 2: Middleware Abstraction & Edge Security

Defending the architecture where Node-RED acts as a middleware buffer between the ESP32 and MongoDB is highly justifiable. Direct edge-to-database communication is widely considered a major security flaw.

Why exposing Port 27017 to the public internet is a massive risk:

- **Massive Attack Surface (The Botnet Target):** Port 27017 is the default port for MongoDB. Malicious bots constantly scan the public internet for exposed default database ports.
- **Credential Compromise on the Edge:** Microcontrollers like the ESP32 are deployed "in the wild." If an attacker physically steals a sensor or extracts its flash memory via a debug port (like JTAG/UART), they can reverse-engineer the firmware. If that firmware contains the direct database connection string, **the attacker now has full read/write access to your entire database cluster**, not just that single sensor's data.
- **Lack of Rate Limiting and Flood Protection:** Databases are built to store and index data, not to defend against Distributed Denial of Service (DDoS) attacks. If an edge device gets caught in an infinite boot-loop or gets hijacked, it could flood the database with millions of queries, exhausting connections and crashing the entire system.